



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Fundamentos Matemáticos de Inteligencia Artificial

Clase 8

Manuel A. Sánchez
2026.05.04

Optimización

- 1 Método de Newton
- 2 Métodos Quasi-Newton
- 3 Método del descenso del gradiente
- 4 Subgradientes
- 5 Método de Levenberg-Marquardt para NLS
- 6 Clasificación: Regresión Logística
- 7 Descenso del Gradiente Estocástico (SGD)
- 8 Optimización con Restricciones
- 9 Minimización cuadrática con restricciones lineales

Método de Newton

Problema no lineal

Buscamos el punto $\mathbf{x}^*$ donde se alcanza el mínimo de una función diferenciable $F : \mathbb{R}^n \rightarrow \mathbb{R}$.

En otras palabras, buscamos $\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x}} F(\mathbf{x})$.

Como la función es diferenciable sabemos que en el óptimo:

$$\nabla F(\mathbf{x}^*) = \mathbf{0}.$$

Propondremos un método iterativo para resolver esta ecuación **no lineal**.

Supongamos que estamos en un punto $\mathbf{x}_k$ cercano a la solución $\mathbf{x}^*$ y queremos movernos a un nuevo punto $\mathbf{x}_{k+1}$ más cerca de la solución.

¿Cómo escogemos el salto $\Delta \mathbf{x}_k$ para llegar a $\mathbf{x}_{k+1}$?

Método de Newton

Cerca del punto $\mathbf{x}_k$, el gradiente ∇F es bien aproximado por su plano tangente usando la matriz Hessiana H :

$$\nabla F(\mathbf{x}_k + \Delta \mathbf{x}_k) \approx \nabla F(\mathbf{x}_k) + H(\mathbf{x}_k) \Delta \mathbf{x}_k$$

Forzando a que el nuevo gradiente sea cero, obtenemos el salto $\Delta \mathbf{x}_k$:

$$\textbf{Método de Newton: } \mathbf{x}_{k+1} = \mathbf{x}_k - [H(\mathbf{x}_k)]^{-1} \nabla F(\mathbf{x}_k)$$

Esto es matemáticamente equivalente a encontrar $\mathbf{x}_{k+1}$ como el mínimo del polinomio de Taylor de orden 2 alrededor de $\mathbf{x}_k$:

$$\mathbf{x}_{k+1} = \underset{\mathbf{x}}{\operatorname{argmin}} \left(F(\mathbf{x}_k) + \nabla F(\mathbf{x}_k)^\top (\mathbf{x} - \mathbf{x}_k) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_k)^\top H(\mathbf{x}_k) (\mathbf{x} - \mathbf{x}_k) \right)$$

Convergencia

Theorem

Si $F : \mathbb{R}^D \rightarrow \mathbb{R}$ es una función dos veces diferenciable, y el punto inicial $\mathbf{x}_0$ está lo suficientemente cerca del óptimo $\mathbf{x}^*$ (convergencia local), entonces la iteración de Newton satisface que:

$$\|\mathbf{x}_{k+1} - \mathbf{x}^*\| \leq C \|\mathbf{x}_k - \mathbf{x}^*\|^2$$

Esto se conoce como **convergencia cuadrática** (el número de dígitos correctos se duplica en cada iteración).

Ejemplo

Considere la función $F(x) = \frac{1}{3}x^3 - 4x$. Entonces:

$$\nabla F(x) = x^2 - 4, \quad H(x) = 2x.$$

La iteración de Newton para un punto x_k queda definida como:

$$x_{k+1} = x_k - \frac{1}{2x_k} (x_k^2 - 4) = \frac{x_k}{2} + \frac{2}{x_k}.$$

Nota: Esta es exactamente la fórmula del antiguo método babilónico para aproximar $\sqrt{4} = 2$.

Método de Newton en la Práctica

El Problema: Converge muy rápido **solo si** estamos cerca del óptimo $\mathbf{x}^*$. Si estamos lejos, el salto dictado por el Hessiano puede enviarnos en la dirección equivocada o demasiado lejos.

Backtracking Line Search

Controlamos la longitud del salto multiplicando la dirección por un paso $t \in (0, 1]$.

Dirección de Newton: $\Delta \mathbf{x}_k = -H(\mathbf{x}_k)^{-1} \nabla F(\mathbf{x}_k)$

Paso inicial: Comenzar asumiendo un paso completo ($t = 1$).

Reducción iterativa: Achicar el paso ($t \leftarrow \beta t$) hasta cumplir la **Condición de Armijo**:

$$F(\mathbf{x}_k + t\Delta \mathbf{x}_k) \leq F(\mathbf{x}_k) + \alpha t \nabla F(\mathbf{x}_k)^\top \Delta \mathbf{x}_k$$

Donde $\beta \in (0, 1)$ es el factor de reducción y $\alpha \in (0, 0.5)$ es la tolerancia exigida.

Métodos Quasi-Newton

El origen: Del Método de Newton a la Secante

El cuello de botella: Newton en múltiples dimensiones requiere calcular y evaluar el Hessiano analítico en cada paso, lo cual es prohibitivo.

Recordemos Cálculo I. Si queremos la raíz de $g(x) = 0$ usando Newton, iteramos $x_{k+1} = x_k - g(x_k)/g'(x_k)$. Si no podemos calcular la derivada $g'(x_k)$, la aproximamos usando los dos últimos puntos (**Método de la Secante**):

$$g'(x_k) \approx \frac{g(x_k) - g(x_{k-1})}{x_k - x_{k-1}}$$

En optimización, buscamos raíces del gradiente ($g = \nabla f$). Así, queremos aproximar la segunda derivada (el Hessiano) usando la diferencia de los primeros gradientes.

La Ecuación de la Secante (Multidimensional)

Definamos los vectores de cambio entre dos iteraciones sucesivas:

$$\mathbf{s}_k = \mathbf{x}_k - \mathbf{x}_{k-1} \quad (\text{Cambio en la posición})$$

$$\mathbf{y}_k = \nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1}) \quad (\text{Cambio en el gradiente})$$

Por el Teorema de Taylor, $\nabla f(\mathbf{x}_k) \approx \nabla f(\mathbf{x}_{k-1}) + H(\mathbf{x}_k)(\mathbf{x}_k - \mathbf{x}_{k-1})$. Así, exigimos que nuestra aproximación del Hessiano, que llamaremos B_k , cumpla la **Ecuación de la Secante**:

Ecuación de la Secante

$$B_k \mathbf{s}_k = \mathbf{y}_k$$

Problema: B_k tiene n^2 incógnitas, pero la ecuación solo nos da n restricciones. ¿Hay infinitas matrices que cumplen esto!

El Método de Broyden

¿Cómo elegimos una única matriz B_k entre las infinitas posibles?

La heurística de Broyden (1965): Elijamos la matriz B_k que cumpla la ecuación de la secante, pero que sea **lo más parecida posible** a nuestra aproximación anterior B_{k-1} .

$$\min_{B_k} \|B_k - B_{k-1}\|_F \quad \text{sujeto a} \quad B_k \mathbf{s}_k = \mathbf{y}_k$$

La solución analítica a este problema es la famosa **Actualización de Broyden (Rango 1)**:

$$B_k = B_{k-1} + \frac{(\mathbf{y}_k - B_{k-1} \mathbf{s}_k) \mathbf{s}_k^\top}{\mathbf{s}_k^\top \mathbf{s}_k}$$

Con esto evitamos calcular el Hessiano, solo lo vamos "actualizando" en cada iteración sumándole una matriz de rango 1.

De Broyden a BFGS

El método de Broyden es genial para sistemas de ecuaciones generales, pero en optimización sabemos que el Hessiano analítico tiene dos propiedades sagradas:

Es **Simétrico** ($H = H^\top$).

Cerca del mínimo, es **Definido Positivo** ($\mathbf{x}^\top H \mathbf{x} > 0$).

¡La actualización de Broyden destruye la simetría! Para arreglarlo, imponemos la simetría como restricción, lo que resulta en una actualización más compleja de **Rango 2**.

Esta es la actualización **BFGS (Broyden-Fletcher-Goldfarb-Shanno)**. Además, usando la fórmula de Sherman-Morrison, podemos actualizar directamente la *inversa* del Hessiano ($H_k \approx B_k^{-1}$), evadiendo el temido costo de $O(n^3)$ para invertir matrices.

El salto a L-BFGS (Memoria Limitada)

BFGS reduce el cálculo computacional a $\mathcal{O}(n^2)$ por iteración. Es un gran avance, pero en Inteligencia Artificial...

El Problema de la Memoria: Si una red neuronal tiene $n = 1,000,000$ de pesos, almacenar la matriz densa H_k requiere 10^{12} números (~ 4 Terabytes de RAM).

La Solución: L-BFGS (*Limited-memory BFGS*)

En lugar de almacenar la inmensa matriz H_k , notamos que ésta se construye sumando iterativamente los vectores $\mathbf{s}_i$ y $\mathbf{y}_i$.

L-BFGS **nunca construye la matriz.**

Solo guarda en memoria los últimos m pares de vectores $(\mathbf{s}_i, \mathbf{y}_i)$ (usualmente $m \approx 10$).

Calcula el producto matriz-vector al vuelo. ¡El costo de memoria baja a $\mathcal{O}(mn)$!

Método del descenso del gradiente

Método del descenso del gradiente

Consideramos nuevamente el problema de minimizar una función $f : \mathbb{R}^D \rightarrow \mathbb{R}$, diferenciable

$$\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x} \in \mathbb{R}^D} f(\mathbf{x})$$

La iteración del método del descenso del gradiente se escribe como:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \gamma_k \nabla f(\mathbf{x}^{(k)})$$

donde γ_k es un parámetro de paso (learning rate) adecuado, de tal forma que la sucesión

$$f(\mathbf{x}^{(0)}) \geq f(\mathbf{x}^{(1)}) \geq f(\mathbf{x}^{(2)}) \geq \dots \geq f(\mathbf{x}^{(n)}) \geq$$

converge a un mínimo local de f .

Búsqueda de línea exacta (*Exact Line Search*)

Es una estrategia para determinar la longitud del paso de manera que minimice la función en la dirección de descenso (en este caso, el gradiente negativo).

En otras palabras, queremos encontrar el tamaño del paso γ_k que nos lleve al punto más bajo posible a lo largo de esa línea recta:

$$\min_{\gamma \geq 0} G(\gamma) := \min_{\gamma \geq 0} f\left(\mathbf{x}^{(k)} - \gamma \nabla f(\mathbf{x}^{(k)})\right)$$

Esto convierte un problema n-dimensional en un problema de optimización de 1 sola variable (γ), que puede resolverse usando métodos de Cálculo I (haciendo $G'(\gamma) = 0$).

Interpretación geométrica

Para el caso $n = 2$ podemos interpretar el método:

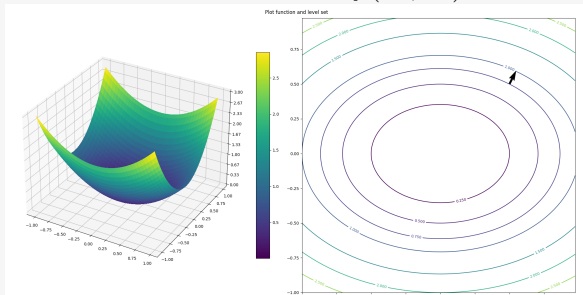
$$\nabla f(x_1, x_2) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right]$$

es la dirección más pronunciada y perpendicular a la curva de nivel $f(x_1, x_2) = \text{cte.}$

Ejemplo.

$$f(x_1, x_2) = ax_1^2 + bx_2^2$$

$$\nabla f(x_1, x_2) = 2ax_1 + 2bx_2$$



Ejemplo Zig-Zag

Considere la función $f(x_1, x_2) = \frac{1}{2}(x_1^2 + bx_2^2)$. La iteración del descenso del gradiente para esta función resulta:

$$\begin{bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \end{bmatrix} = \begin{bmatrix} x_1^{(k)} \\ x_2^{(k)} \end{bmatrix} - \gamma_k \begin{bmatrix} x_1^{(k)} \\ bx_2^{(k)} \end{bmatrix}$$

Exact line search

$$\min_{\gamma_k} f(x_1^{(k+1)}, x_2^{(k+1)}) = \min_{\gamma_k} \frac{1}{2} \left((x_1^{(k)} - \gamma_k x_1^{(k)})^2 + b(x_2^{(k)} - \gamma_k bx_2^{(k)})^2 \right) =: \min_{\gamma_k} G(\gamma_k)$$

$$\text{Entonces: } G'(\gamma_k) = 0 \longrightarrow -x_1^{(k)}(1 - \gamma_k) - b^2 x_2^{(k)}(1 - b\gamma_k) = 0$$

$$\text{De donde: } \gamma_k = \frac{(x_1^{(k)})^2 + (bx_2^{(k)})^2}{(x_1^{(k)})^2 + b^3(x_2^{(k)})^2}$$

Ejemplo Zig-Zag

Comenzando la iteración con el punto $(x_1^{(0)}, x_2^{(0)}) = (b, 1)$ obtenemos $\gamma_0 = \frac{2}{1+b}$ y así

$$x_1^{(1)} = b \left(\frac{b-1}{b+1} \right), \quad x_2^{(1)} = \left(\frac{b-1}{b+1} \right)$$

Para la siguiente iteración calculamos $\gamma_1 = \frac{2}{1+b}$ y así

$$x_1^{(2)} = b \left(\frac{b-1}{b+1} \right)^2, \quad x_2^{(2)} = \left(\frac{b-1}{b+1} \right)^2$$

Calculando sucesivamente obtenemos $\gamma_k = \frac{2}{1+b}$ y

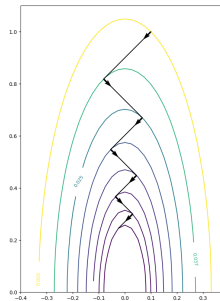
$$x_1^{(k)} = b \left(\frac{b-1}{b+1} \right)^k, \quad x_2^{(k)} = \left(\frac{b-1}{b+1} \right)^k$$

Ejemplo Zig-Zag

Además calculamos el valor de la función de pérdida

$$f(x_1^{(k)}, x_2^{(k)}) = \left(\frac{1-b}{1+b}\right)^{2k} f(x_1^{(0)}, x_2^{(0)}) = \left(\frac{1-b}{1+b}\right)^{2k} \frac{1}{2}(b^2 + b)$$

Observe que si b es cercano a 0, la iteración disminuye lentamente.



Análisis del método de descenso de gradiente

Asuma que f es una función estrictamente convexa y dos veces diferenciable, y además asuma que existen constantes $m, M > 0$ tales que:

$$mI \leq H(x) \leq MI; \quad H_{ij}(x) = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

Sea la función $G(\gamma_k) = f(x^{(k)} - \gamma_k \nabla f(x^{(k)}))$, entonces:

$$G(\gamma_k) \leq f(x^{(k)}) - \gamma \|\nabla f(x^{(k)})\|_2^2 + \frac{M\gamma^2}{2} \|\nabla f(x^{(k)})\|_2^2$$

El mínimo se alcanza para $\gamma = 1/M$, así

$$f(x^{(k+1)}) \leq f(x^{(k)}) - \frac{1}{2M} \|\nabla f(x^{(k)})\|_2^2$$

Análisis del Descenso del Gradiente

Por otro lado, usando la cota inferior $f(x^*) \geq f(\mathbf{x}^{(k)}) - \frac{1}{2m} \|\nabla f(\mathbf{x}^{(k)})\|_2^2$, concluimos que:

$$f(\mathbf{x}^{(k+1)}) - f(x^*) \leq \left(1 - \frac{m}{M}\right) (f(\mathbf{x}^{(k)}) - f(x^*))$$

El Número de Condición (κ)

Definimos el número de condición como $\kappa = M/m$ (la razón entre la máxima y la mínima curvatura).

Decae constantemente el valor de f , logrando **convergencia lineal**.

El Problema: Si κ es muy grande ($M \gg m$), entonces $(1 - 1/\kappa)$ es muy cercano a 1. ¡La convergencia será desesperantemente lenta! El valle es tan estrecho que el algoritmo rebota en las paredes.

Búsqueda de línea inexacta (Backtracking)

Al igual que vimos en el Método de Newton, hacer una *Exact Line Search* analítica suele ser imposible para funciones generales $f(\mathbf{x})$.

Usamos nuevamente **Backtracking** para asegurar un descenso suficiente en la dirección del gradiente:

Comenzar con un paso completo (ej. $\gamma = 1$).

Testear la **Condición de Armijo** en la dirección del gradiente:

$$f(\mathbf{x}^{(k)} - \gamma \nabla f(\mathbf{x}^{(k)})) \leq f(\mathbf{x}^{(k)}) - \alpha \gamma \|\nabla f(\mathbf{x}^{(k)})\|^2$$

Si se satisface, aceptamos el salto $\mathbf{x}^{(k+1)}$.

Si no, *backtrack*: reducimos $\gamma \leftarrow \beta \gamma$ (ej. reducimos a la mitad $\beta = 0.5$) y volvemos a evaluar.

Búsqueda de línea inexacta

Algorithm 1: Backtracking

Input : $\mathbf{x}, f(\mathbf{x}), \nabla f(\mathbf{x})$.

Output: Siguiente iteración.

Escoger α, β : $0 < \alpha < 0.5, 0 < \beta < 1$;

Inicializar $\gamma = 1$;

$X = \mathbf{x} - \gamma \nabla f(\mathbf{x})$;

while $f(X) > f(\mathbf{x}) - \alpha \gamma \|\nabla f(\mathbf{x})\|^2$ **do**

$\gamma = \beta \gamma$;

$X = \mathbf{x} - \gamma \nabla f(\mathbf{x})$;

end

return X

Análisis del método de desc. de gradiente-backtracking

Si suponemos que $0 \leq \gamma_k \leq 1/M$, entonces:

$$G(\gamma_k) \leq f(x_k) - \frac{1}{2}\gamma_k \|\nabla f(x_k)\|_2^2 \leq f(x_k) - \alpha\gamma_k \|\nabla f(x_k)\|_2^2, \quad \alpha < \frac{1}{2}$$

Se sigue que backtracking termina con $\gamma = 1$ o $\gamma \geq \beta/M$, escribimos

$$f(x_{k+1}) \leq f(x_k) - \min\{\alpha, \beta\alpha/M\} \|\nabla f(x_k)\|_2^2.$$

Así, podemos concluir tal como en el caso anterior

$$f(x_{k+1}) - f(x^*) \leq (1 - \min\{\alpha, \beta, \alpha/M\})(f(x_k) - f(x^*))$$

Descenso con momentum

Consideremos nuevamente el problema modelo $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + bx_2^2)$. Su matriz Hessiana es diagonal con entradas 1 y b , el cual tiene un problema cuando $b = \lambda_{\min}/\lambda_{\max} = m/M$ es pequeño.

El comportamiento zig-zag no sucedería para una bola pesada rodando cuesta abajo. Su momentum lo lleva hacia valle estrecho. Así añadimos momentum con un coeficiente β al gradiente (Polyak). La dirección $\mathbf{z}^{(k)}$ del nuevo paso recuerda la dirección previa $\mathbf{z}^{(k-1)}$.

Descenso con momentum: $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \gamma \mathbf{z}^{(k)}, \quad \mathbf{z}^{(k)} = \nabla f(\mathbf{x}^{(k)}) + \beta \mathbf{z}^{(k-1)}.$

Observe que ahora que el paso $\mathbf{x}^{(k+1)}$ involucra a $\mathbf{z}^{(k-1)}$, es decir, es un método de dos pasos.

Descenso con momentum

El método puede ser escrito como un método de paso simple como uno de dos ecuaciones

$$\text{Descenso con momentum:} \quad \begin{cases} \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} - \gamma \mathbf{z}^{(k)} \\ \mathbf{z}^{(k+1)} - \nabla f(\mathbf{x}^{(k+1)}) &= \beta \mathbf{z}^{(k)} \end{cases}$$

Ejemplo: el modelo cuadrático.

Sea $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top S\mathbf{x}$, con S simétrica y definida positiva $\nabla f(\mathbf{x}) = S\mathbf{x}$. Seguimos los vectores propios de S . Suponga que:

$$S\mathbf{q} = \lambda\mathbf{q}, \quad \mathbf{x}^{(k)} = c_k\mathbf{q}, \quad \mathbf{z}^{(k)} = d_k\mathbf{q}, \quad \nabla f(\mathbf{x}_k) = S\mathbf{x}_k = \lambda c_k\mathbf{q}$$

Así nuestra ecuación para el descenso con momentum nos permite escribir

$$\left. \begin{aligned} c_{k+1} &= c_k - \gamma d_k \\ -\lambda c_{k+1} + d_{k+1} &= \beta d_k \end{aligned} \right\} \longleftrightarrow \begin{bmatrix} 1 & 0 \\ -\lambda & 1 \end{bmatrix} \begin{bmatrix} c_{k+1} \\ d_{k+1} \end{bmatrix} = \begin{bmatrix} 1 & -\gamma \\ 0 & \beta \end{bmatrix} \begin{bmatrix} c_k \\ d_k \end{bmatrix}$$

Invirtiendo el sistema, obtenemos el paso de descenso:

$$\begin{bmatrix} c_{k+1} \\ d_{k+1} \end{bmatrix} = \begin{bmatrix} 1 & -\gamma \\ \lambda & \beta - \lambda\gamma \end{bmatrix} \begin{bmatrix} c_k \\ d_k \end{bmatrix} = R \begin{bmatrix} c_k \\ d_k \end{bmatrix}$$

Ejemplo: el modelo cuadrático.

Para una convergencia rápida, necesitamos que los valores propios de R , e_1 y e_2 , sean lo mas pequeños posible. Estos dependen del valor propio λ de S . Así, necesitamos escoger γ y β que minimizar:

$$\max\{|e_1(\lambda)|, |e_2(\lambda)|\}, \quad \text{para } \lambda_{\min}(S) \leq \lambda \leq \lambda_{\max}(S).$$

Para este ejemplo, la solución es:

$$\gamma = \left(\frac{2}{\sqrt{\lambda_{\max}} + \sqrt{\lambda_{\min}}} \right)^2, \quad \beta = \left(\frac{\sqrt{\lambda_{\max}} - \sqrt{\lambda_{\min}}}{\sqrt{\lambda_{\max}} + \sqrt{\lambda_{\min}}} \right)^2$$

Ejemplo: el problema zig-zag

Si consideramos este resultado al problema modelo $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + bx_2^2)$, tenemos que los valores propios de S son $\lambda_{\max} = 1$, $\lambda_{\min} = b$, así

$$\gamma = \left(\frac{2}{1 + \sqrt{b}} \right)^2, \quad \beta = \left(\frac{1 - \sqrt{b}}{1 + \sqrt{b}} \right)^2$$

Así pasamos, en términos del facto de descenso de

$$\underbrace{\left(\frac{1-b}{1+b} \right)^2}_{\text{desc. del gradiente}} \quad \longrightarrow \quad \underbrace{\left(\frac{1-\sqrt{b}}{1+\sqrt{b}} \right)^2}_{\text{desc. del gradiente acelerado}}$$

Ejemplo: La Magia de Momentum (Problema Zig-Zag)

Si aplicamos la solución óptima de γ y β a nuestro problema modelo con $b = \lambda_{\min}/\lambda_{\max}$, el factor de contracción del error pasa a ser:

$$\underbrace{\left(\frac{1-b}{1+b}\right)^2}_{\text{Descenso del Gradiente}} \longrightarrow \underbrace{\left(\frac{1-\sqrt{b}}{1+\sqrt{b}}\right)^2}_{\text{Gradiente con Momentum}}$$

La diferencia en la práctica

Imagina un valle muy mal condicionado donde $b = 0.01$ (la pendiente en una dirección es 100 veces mayor que en la otra).

Gradiente Clásico: $(0.99/1.01)^2 \approx \mathbf{0.96}$

Momentum: $(0.9/1.1)^2 \approx \mathbf{0.67}$

¡Momentum reduce drásticamente el número de iteraciones necesarias para converger al transformar el número de condición κ en $\sqrt{\kappa}$!

Nesterov Accelerated Gradient (NAG)

Intuición: "Mirar antes de saltar".

Momentum Clásico: Primero calcula el gradiente y luego suma el momentum acumulado.

Nesterov: Primero hace un "salto de fe" siguiendo el momentum, y calcula el gradiente en esa nueva posición.

$$\begin{aligned}\mathbf{v}^{(k+1)} &= \beta \mathbf{v}^{(k)} + \gamma \nabla f(\mathbf{x}^{(k)} + \beta \mathbf{v}^{(k)}) \\ \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} - \mathbf{v}^{(k+1)}\end{aligned}$$

Esto evita que el algoritmo "se pase de largo" en curvas cerradas, mejorando drásticamente la estabilidad en redes neuronales profundas.

Subgradiantes

Optimización de Funciones No Suaves)

Queremos optimizar funciones convexas que no son diferenciables en todos sus puntos (tienen "puntas").

Recordemos la condición de primer orden para funciones **convexas** diferenciables

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}), \quad \forall \mathbf{x}, \mathbf{y}$$

Esto significa que la línea tangente siempre está por debajo de la gráfica de la función.

Definición de Subgradiente:

Si f es convexa pero no necesariamente diferenciable en $\mathbf{x}$, un vector $\mathbf{g}$ es un **subgradiente** de f en $\mathbf{x}$ si satisface:

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \mathbf{g}^\top (\mathbf{y} - \mathbf{x}), \quad \text{para todo } \mathbf{y}$$

Al conjunto de todos los subgradientes posibles en $\mathbf{x}$ se le llama **subdiferencial** y se denota $\partial f(\mathbf{x})$.

Ejemplo 1: Valor Absoluto (Regularización Lasso)

Sea $f(x) = |x|$. Sabemos que es diferenciable en todo $\mathbb{R} \setminus \{0\}$.

$$\text{Si } x > 0 \implies \nabla f(x) = 1 \implies \partial f(x) = \{1\}$$

$$\text{Si } x < 0 \implies \nabla f(x) = -1 \implies \partial f(x) = \{-1\}$$

¿Qué pasa en $x = 0$? Buscamos los valores g que cumplan la definición:

$$f(y) \geq f(0) + g(y - 0) \implies |y| \geq gy, \quad \text{para todo } y$$

Esta desigualdad se cumple si y sólo si $-1 \leq g \leq 1$. Por lo tanto, la subdiferencial en el origen es el intervalo:

$$\partial f(0) = [-1, 1]$$

Cualquier línea que pase por el origen con pendiente entre -1 y 1 se mantendrá siempre por debajo de $|x|$.

Ejemplo 2: Función ReLU (Redes Neuronales)

Sea la función de activación *Rectified Linear Unit*, $f(x) = \max(0, x)$.

$$\text{Si } x > 0 \implies \partial f(x) = \{1\}$$

$$\text{Si } x < 0 \implies \partial f(x) = \{0\}$$

En la "punta" $x = 0$, cualquier línea con pendiente entre 0 y 1 servirá como un piso válido para la función.

$$\partial f(0) = [0, 1]$$

Conexión con IA: Backpropagation

Cuando entrenamos una red neuronal y una neurona ReLU recibe exactamente $x = 0$, matemáticamente hay infinitos gradientes válidos. En la práctica, librerías como PyTorch o TensorFlow están programadas para escoger arbitrariamente $g = 0$ (a veces $g = 0.5$) de la subdiferencial, para poder continuar aplicando la Regla de la Cadena.

Método del Subgradiente

Si nuestra función de pérdida $L(\mathbf{x})$ no es diferenciable, podemos reemplazar el Descenso del Gradiente Clásico por el **Método del Subgradiente**:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \gamma_k \mathbf{g}^{(k)}, \quad \text{donde } \mathbf{g}^{(k)} \in \partial L(\mathbf{x}^{(k)})$$

Solo necesitamos calcular un subgradiente cualquiera en cada paso.

Precaución: A diferencia del gradiente tradicional, moverse en la dirección del subgradiente negativo **no garantiza** que la función disminuya en cada paso (no es un método de descenso estricto).

Por esta razón, mantenemos un registro del mejor valor f_{best} visto hasta ahora y requerimos tasas de aprendizaje γ_k decrecientes.

Método de Levenberg-Marquardt para NLS

Mínimos Cuadrados No Lineales (NLS)

En muchas aplicaciones de Inteligencia Artificial y Computación Científica, nuestro objetivo no es clasificar, sino ajustar un modelo no lineal $\hat{y} = f(\mathbf{x}; \boldsymbol{\theta})$ a un conjunto de m observaciones $(\mathbf{x}_i, y_i)$.

Buscamos los parámetros óptimos $\boldsymbol{\theta} = [\theta_1, \dots, \theta_n]^\top$ que minimicen la función de pérdida cuadrática (**Square Loss**):

$$L(\boldsymbol{\theta}) = \frac{1}{2} \sum_{i=1}^m (y_i - f(\mathbf{x}_i; \boldsymbol{\theta}))^2 = \frac{1}{2} \|\mathbf{y} - \hat{\mathbf{y}}(\boldsymbol{\theta})\|_2^2$$

Para minimizar, calculamos el gradiente respecto a $\boldsymbol{\theta}$ e igualamos a cero:

$$\nabla L(\boldsymbol{\theta}) = -J^\top (\mathbf{y} - \hat{\mathbf{y}}(\boldsymbol{\theta})) = \mathbf{0}$$

donde $J \in \mathbb{R}^{m \times n}$ es la **Matriz Jacobiana** (derivadas de las predicciones $\hat{y}_i$ respecto a los parámetros θ_j).

El Truco de Gauss-Newton: Evadir el Hessiano

Si intentáramos usar el **Método de Newton** clásico para resolver esto, necesitaríamos el Hessiano H , lo cual incluye calcular segundas derivadas horribles de nuestro modelo $f(\mathbf{x}; \boldsymbol{\theta})$.

Sin embargo, para problemas de mínimos cuadrados, la teoría nos dice que cerca del mínimo, el Hessiano se puede aproximar usando **sólo el Jacobiano** (información de primer orden):

$$H \approx J^T J$$

Sustituyendo esto en la fórmula de Newton ($\Delta\boldsymbol{\theta} = -H^{-1}\nabla L$), obtenemos el **Método de Gauss-Newton**:

$$(J^T J)\Delta\boldsymbol{\theta} = J^T(\mathbf{y} - \hat{\mathbf{y}}(\boldsymbol{\theta}))$$

¡Logramos convergencia casi cuadrática sin calcular ni una sola segunda derivada!

Hacia LM: Los dos extremos de la Optimización

Para actualizar nuestros parámetros, definamos el salto $\Delta\theta = \theta^{(k+1)} - \theta^{(k)}$ y el vector de error (residual) $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}(\theta^{(k)})$.

Tenemos dos métodos extremos para encontrar $\Delta\theta$:

1. Descenso del Gradiente Clásico (Seguro pero lento)

$$\begin{aligned}\Delta\theta &= -\gamma \nabla L(\theta^{(k)}) = \gamma J^\top \mathbf{e} \\ \Rightarrow \quad \left(\frac{1}{\gamma} I\right) \Delta\theta &= J^\top \mathbf{e}\end{aligned}$$

2. Gauss-Newton / Aproximación de Newton (Rápido pero inestable)

Asumiendo $H \approx J^\top J$, el paso de Newton ($-H^{-1} \nabla L$) se escribe:

$$(J^\top J) \Delta\theta = J^\top \mathbf{e}$$

Método de Levenberg-Marquardt

Levenberg-Marquardt (LM) fusiona ambas ecuaciones sumando sus matrices, controladas por un parámetro de amortiguamiento (*damping*) $\lambda \geq 0$:

Actualización LM

$$(J^T J + \lambda I) \Delta \theta = J^T \mathbf{e}$$

El algoritmo ajusta λ dinámicamente durante el entrenamiento, transitando suavemente entre dos fases:

Fase 1: Exploración (Comportamiento de Gradiente).

Fase 2: Explotación (Comportamiento de Newton).

Las Dos Fases de Levenberg-Marquardt

Fase 1: Lejos del óptimo (Inicio del algoritmo)

Se utiliza un λ grande.

El término diagonal domina fuertemente la matriz: $J^T J + \lambda I \approx \lambda I$.

La ecuación se reduce a: $\lambda I \Delta \theta \approx J^T \mathbf{e} \implies \Delta \theta \approx \frac{1}{\lambda} J^T \mathbf{e}$.

Efecto: Es idéntico al **Descenso del Gradiente** con una tasa de aprendizaje pequeña $\gamma = 1/\lambda$. Es un paso robusto y seguro.

Fase 2: Cerca del óptimo (Final del algoritmo)

La aproximación cuadrática se vuelve confiable, así que reducimos $\lambda \rightarrow 0$.

El término de la identidad desaparece: $J^T J + \lambda I \approx J^T J$.

La ecuación se reduce a: $(J^T J) \Delta \theta \approx J^T \mathbf{e}$.

Efecto: Es idéntico a **Gauss-Newton**, logrando convergencia rápida (casi cuadrática) directo hacia el mínimo.

Clasificación: Regresión Logística

Ejemplo: Regresión logística

Consideremos una variable aleatoria $Y \in \{0, 1\}$ con:

$$P(Y = 1) = p, \quad P(Y = 0) = 1 - p, \quad p \in [0, 1].$$

Ilustración. $Y = 1$ puede ser la probabilidad de que un individuo adquiriera cierta enfermedad. La probabilidad p de tener la enfermedad se modela como función de un vector de características $\mathbf{x} \in \mathbb{R}^n$, el cual puede representar el peso, edad, altura, presión arterial, u otras variables clínicas relevantes.

El Modelo Logístico

Para asegurar que la probabilidad esté siempre en el rango $(0, 1)$, el modelo logístico toma la forma de la función sigmoide:

$$p_i = P(Y = 1|\mathbf{x}_i) = \frac{\exp(\mathbf{w}^\top \mathbf{x}_i + b)}{1 + \exp(\mathbf{w}^\top \mathbf{x}_i + b)}, \quad \mathbf{w} \in \mathbb{R}^n, b \in \mathbb{R}.$$

Notemos que la probabilidad contraria es $1 - p_i = \frac{1}{1 + \exp(\mathbf{w}^\top \mathbf{x}_i + b)}$.

Suponga que tenemos un conjunto de datos observados de N pacientes: las variables $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^n$ junto con sus respectivos diagnósticos $y_1, \dots, y_N \in \{0, 1\}$. Nuestro objetivo es encontrar los parámetros óptimos $\mathbf{w}$ y b .

Derivación: De Bernoulli a la Verosimilitud

Dado que nuestra etiqueta es binaria ($y_i \in \{0, 1\}$), podemos escribir la probabilidad de observar la etiqueta correcta para un solo paciente usando el “truco” de la distribución de Bernoulli en una sola línea:

$$P(Y = y_i | \mathbf{x}_i) = p_i^{y_i} (1 - p_i)^{1-y_i}$$

Si $y_i = 1$, el exponente de $(1 - p_i)$ es cero y el término se reduce a p_i .

Si $y_i = 0$, el exponente de p_i es cero y el término se reduce a $(1 - p_i)$.

Asumiendo que nuestras N observaciones son independientes, la probabilidad conjunta de observar **todo** el conjunto de datos es el producto de las probabilidades individuales. Esta es la **Función de Verosimilitud** ($\mathcal{L}$):

$$\mathcal{L}(\mathbf{w}, b) = \prod_{i=1}^N p_i^{y_i} (1 - p_i)^{1-y_i}$$

Derivación: Log-Verosimilitud Negativa

Por el principio de Máxima Verosimilitud, queremos **maximizar** $\mathcal{L}(\mathbf{w}, b)$.

Sin embargo, multiplicar miles de probabilidades causa *underflow* numérico en la computadora. Aplicamos la función $\log$ para transformar el producto en una sumatoria:

$$\begin{aligned}\log \mathcal{L}(\mathbf{w}, b) &= \log \left(\prod_{i=1}^N p_i^{y_i} (1 - p_i)^{1-y_i} \right) = \sum_{i=1}^N \left[\log(p_i^{y_i}) + \log((1 - p_i)^{1-y_i}) \right] \\ &= \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]\end{aligned}$$

Como el Descenso del Gradiente está diseñado para **minimizar**, multiplicamos por -1 para obtener la **Log-Verosimilitud Negativa** (o *Cross-Entropy Loss*):

$$L(\mathbf{w}, b) = - \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Simplificación de la Función de Pérdida

Para derivar el gradiente, primero simplificamos $L(\mathbf{w}, b)$. Definamos $z_i = \mathbf{w}^\top \mathbf{x}_i + b$. Sabemos que la sigmoide nos da:

$$\begin{aligned}\log(p_i) &= \log\left(\frac{\exp(z_i)}{1 + \exp(z_i)}\right) = z_i - \log(1 + \exp(z_i)) \\ \log(1 - p_i) &= \log\left(\frac{1}{1 + \exp(z_i)}\right) = -\log(1 + \exp(z_i))\end{aligned}$$

Reemplazando estos valores en nuestra función de pérdida:

$$\begin{aligned}L(\mathbf{w}, b) &= -\sum_{i=1}^N [y_i(z_i - \log(1 + \exp(z_i))) - (1 - y_i)\log(1 + \exp(z_i))] \\ &= -\sum_{i=1}^N [y_i z_i - y_i \log(1 + \exp(z_i)) - \log(1 + \exp(z_i)) + y_i \log(1 + \exp(z_i))]\end{aligned}$$

Simplificación de la Función de Pérdida

¡Los términos cruzados se cancelan mágicamente!

$$L(\mathbf{w}, b) = \sum_{i=1}^N \left[\log(1 + \exp(\mathbf{w}^\top \mathbf{x}_i + b)) - y_i(\mathbf{w}^\top \mathbf{x}_i + b) \right]$$

Observemos detenidamente nuestra función de pérdida simplificada:

El término $-y_i z_i$ es una función afín (lineal), por lo tanto, es convexa.

El término $\log(1 + \exp(z_i))$ se conoce en IA como la función **Softplus**. Su segunda derivada respecto a z_i es exactamente la varianza de la predicción: $p_i(1 - p_i) > 0$. Por ende, es **estrictamente convexa**.

Cálculo del Gradiente

Derivamos $L(\mathbf{w}, b)$ respecto al sesgo b , usando la regla de la cadena:

$$\begin{aligned}\frac{\partial L}{\partial b} &= \sum_{i=1}^N \left[\frac{\exp(\mathbf{w}^\top \mathbf{x}_i + b)}{1 + \exp(\mathbf{w}^\top \mathbf{x}_i + b)} \cdot 1 - y_i \right] \\ &= \sum_{i=1}^N (p_i - y_i)\end{aligned}$$

Derivamos respecto al vector de pesos $\mathbf{w}$ obteniendo un resultado análogo:

$$\begin{aligned}\nabla_{\mathbf{w}} L &= \sum_{i=1}^N \left[\frac{\exp(\mathbf{w}^\top \mathbf{x}_i + b)}{1 + \exp(\mathbf{w}^\top \mathbf{x}_i + b)} \mathbf{x}_i - y_i \mathbf{x}_i \right] \\ &= \sum_{i=1}^N (p_i - y_i) \mathbf{x}_i\end{aligned}$$

Descenso del Gradiente

La iteración del Descenso del Gradiente (*Full-Batch*) para actualizar los parámetros es:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \gamma_k \sum_{i=1}^N (p_i^{(k)} - y_i) \mathbf{x}_i$$
$$b^{(k+1)} = b^{(k)} - \gamma_k \sum_{i=1}^N (p_i^{(k)} - y_i)$$

El Problema en la Era del Big Data:

Para dar **un solo paso** hacia el mínimo (una sola iteración k), necesitamos calcular la predicción $p_i^{(k)}$ y realizar la suma vectorial sobre **todos los N ejemplos** del dataset. Si nuestro dataset tiene $N = 1,000,000$ de registros clínicos, requerimos un millón de evaluaciones exponenciales y productos punto *sólo para actualizar los pesos una vez*. El método clásico colapsa.

Descenso del Gradiente Estocástico (SGD)

Descenso del gradiente clásico

Supongamos que queremos minimizar una función de costo de la forma:

$$L(x) = \sum_{i=1}^m \ell_i(x)$$

usando la iteración del descenso de gradiente: $x_{k+1} = x_k - \gamma_k \nabla L(x_k)$. **Posibles problemas de la iteración.**

- 1 Calcular ∇L en cada paso, esto es calcular las derivadas de L con respecto a los pesos x puede ser demasiado costoso. La pérdida total adiciona las pérdidas individuales $\ell(x, v_i)$ para cada muestra v_i en el conjunto de entrenamiento.
- 2 El número de pesos es incluso mas grande. Así $\nabla L = 0$ puede ocurrir para muchos $x^* = 0$. Algunos de estos pueden dar resultados insatisfactorios para datos aún no vistos, así la función de aprendizaje falla en generalizar.

El Concepto de “Estocástico”

En optimización, el término **estocástico** se refiere a la introducción de aleatoriedad para aproximar un valor exacto.

Gradiente Determinista (Full-Batch): Calcula el gradiente exacto usando **todos** los datos.

Gradiente Estocástico: Estima el gradiente real usando solo una **muestra aleatoria** (o un pequeño grupo).

¿Por qué abandonar la exactitud?

En la era del Big Data, con conjuntos de datos de millones de ejemplos ($N \rightarrow \infty$), calcular el gradiente exacto es computacionalmente imposible en cada paso. SGD permite aprender de forma incremental, mucho antes de haber procesado todo el dataset.

De Full-Batch a SGD

Sea nuestra función de pérdida total el promedio de las pérdidas individuales:

$$L(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \ell_i(\boldsymbol{\theta}, \mathbf{x}_i, y_i)$$

Iteración del Descenso de Gradiente Estocástico ($B = 1$): En cada paso k , elegimos un índice $i \in \{1, \dots, N\}$ al azar y actualizamos:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \gamma_k \nabla \ell_i(\boldsymbol{\theta}^{(k)})$$

Ventaja clave: El costo de una iteración es $\mathcal{O}(1)$ respecto al tamaño del dataset N .
¡Es independiente de si tenemos mil o mil millones de datos!

Mini-Batch SGD

Usar una sola muestra ($B = 1$, *Stochastic puro*) genera demasiada varianza en la dirección del gradiente y desaprovecha la arquitectura paralela del hardware moderno (GPUs).

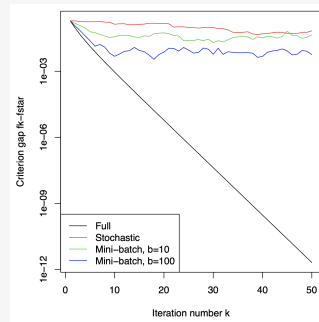
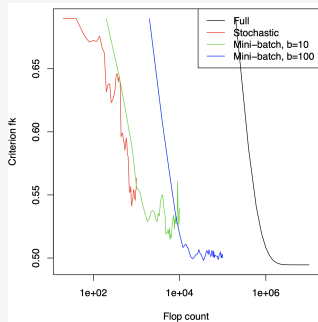
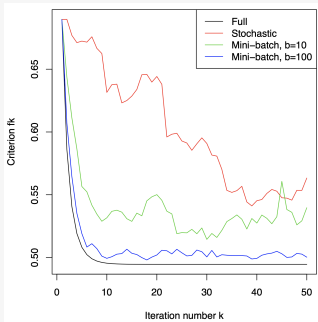
La solución es agrupar un número intermedio de muestras llamado **Mini-batch** de tamaño B (típicamente 32, 64, o 256):

Iteración Mini-Batch SGD

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \gamma_k \left(\frac{1}{B} \sum_{i \in \mathcal{B}_k} \nabla \ell_i(\boldsymbol{\theta}^{(k)}) \right)$$

donde $\mathcal{B}_k$ es un subconjunto elegido aleatoriamente del dataset $\{1, \dots, N\}$ de tamaño B .

Mini-Batch SGD



El Trade-off de Mini-Batch

¿Por qué Mini-Batch es el estándar en IA?

Eficiencia de Hardware (GPUs): Las tarjetas gráficas están diseñadas para multiplicar matrices grandes en paralelo. Calcular el gradiente de $B = 32$ muestras simultáneamente toma casi el mismo tiempo que calcular sólo 1.

Reducción de Varianza: Al promediar B gradientes estocásticos, reducimos la varianza (el "ruido" direccional) por un factor de $1/\sqrt{B}$. El camino hacia el mínimo es mucho más suave y estable que con $B = 1$.

Diccionario Práctico de Machine Learning:

Iteración (*Step*): Una única actualización de los pesos θ (implica procesar 1 mini-batch de tamaño B).

Época (*Epoch*): Un pase completo por los N datos del dataset. Equivale a realizar N/B iteraciones.

Ejemplos de Funciones de Pérdida (ℓ_i)

Para aplicar SGD, la pérdida total debe poder separarse como la suma de las pérdidas individuales $\ell_i(\boldsymbol{\theta})$ para cada muestra $(\mathbf{x}_i, y_i)$.

Error Cuadrático (Square Loss): Usado en regresión continua.

$$\ell_i(\boldsymbol{\theta}) = \frac{1}{2} \|y_i - f(\mathbf{x}_i; \boldsymbol{\theta})\|_2^2$$

Entropía Cruzada (Cross-Entropy): Usado en clasificación (Regresión Logística y Redes Neuronales).

$$\ell_i(\boldsymbol{\theta}) = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Donde $y_i \in \{0, 1\}$ y $\hat{y}_i \in (0, 1)$ es la probabilidad predicha.

Pérdida Bisagra (Hinge Loss): Usado en clasificación de Máquinas de Vectores de Soporte (SVM).

$$\ell_i(\boldsymbol{\theta}) = \max\{0, 1 - y_i f(\mathbf{x}_i; \boldsymbol{\theta})\}$$

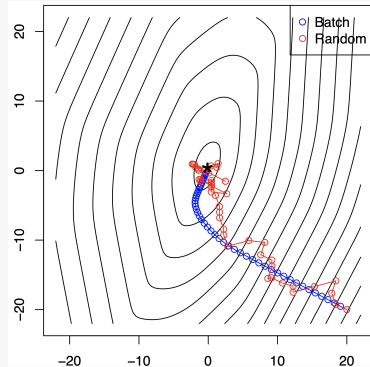
Donde las etiquetas son $y_i \in \{-1, 1\}$.

¿Por qué funciona SGD? (Justificación Matemática)

Aunque el gradiente de una sola muestra $\nabla \ell_i$ es ruidoso y no apunta exactamente al mínimo global, es un **estimador insesgado** del gradiente real.

Si el índice j se elige uniformemente al azar, la esperanza del gradiente estocástico es:

$$\begin{aligned} E[\nabla \ell_j(\boldsymbol{\theta})] &= \sum_{i=1}^N \nabla \ell_i(\boldsymbol{\theta}) P(j = i) \\ &= \sum_{i=1}^N \nabla \ell_i(\boldsymbol{\theta}) \frac{1}{N} \\ &= \nabla L(\boldsymbol{\theta}) \end{aligned}$$



Esto garantiza que, aunque el camino sea ruidoso (“zig-zagueante”), el algoritmo se mueve en la dirección correcta en promedio.

Mejoras Modernas al SGD

Aunque SGD es el motor de la Inteligencia Artificial, sufre en superficies mal condicionadas (valles estrechos) y requiere un ajuste manual cuidadoso de la tasa de aprendizaje γ_k .

Para solucionar esto, la industria utiliza variantes avanzadas:

SGD con Momentum: Al igual que vimos en GD, acumula una fracción de los gradientes pasados. Esto actúa como un amortiguador que **suaviza el ruido** estocástico y acelera el paso en valles consistentes.

$$\mathbf{v}^{(k+1)} = \beta \mathbf{v}^{(k)} + \nabla \ell_{i_k}(\boldsymbol{\theta}^{(k)}) \implies \boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \gamma \mathbf{v}^{(k+1)}$$

Tasas Adaptativas (AdaGrad, RMSProp): En lugar de usar un solo γ_k global, ajustan la tasa de aprendizaje **por cada parámetro**. Los pesos con gradientes históricamente grandes reciben pasos más pequeños, y viceversa.

Adam (Adaptive Moment Estimation): El optimizador por defecto hoy en día. Es la combinación definitiva: calcula una media móvil del gradiente (Momentum) y una media móvil del gradiente al cuadrado (RMSProp).

Convergencia y Condiciones de Robbins-Monro

Debido al ruido intrínseco, SGD no se detiene naturalmente en el mínimo; suele “orbitar” a su alrededor. Para forzar la convergencia, la tasa de aprendizaje γ_k debe decaer cumpliendo:

$\sum_{k=1}^{\infty} \gamma_k = \infty$: Los pasos deben ser suficientemente largos para alcanzar el mínimo desde cualquier punto.

$\sum_{k=1}^{\infty} \gamma_k^2 < \infty$: El ruido (varianza) debe desaparecer eventualmente para que el algoritmo se asiente.

Propiedad Inesperada: Generalización

Se ha observado empíricamente que el ruido de SGD ayuda a las Redes Neuronales a escapar de mínimos locales pobres y encontrar soluciones que **generalizan mejor** a datos nuevos, a diferencia del descenso de gradiente exacto.

Optimización con Restricciones

El Problema General de Optimización

Consideremos el problema de encontrar el mínimo de una función objetivo sujeta a un conjunto de restricciones que definen la región permitida para la solución.

Formulación Matemática:

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^n} f(\boldsymbol{\theta})$$

sujeito a:

Igualdades: $h_i(\boldsymbol{\theta}) = 0, \quad i = 1, \dots, p$

Desigualdades: $g_j(\boldsymbol{\theta}) \leq 0, \quad j = 1, \dots, m$

La región $\mathcal{C} = \{\boldsymbol{\theta} : h_i(\boldsymbol{\theta}) = 0, g_j(\boldsymbol{\theta}) \leq 0\}$ se denomina el **conjunto factible**. Si $\mathcal{C}$ es vacío, el problema no tiene solución.

Ejemplo: Support Vector Machines (SVM)

El objetivo de una SVM es encontrar el hiperplano que maximiza el margen de separación entre dos clases.

Formulación:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

sujeito a las restricciones de margen:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, N$$

Reescribiendo como desigualdades estándar ($g_j \leq 0$):

$$g_i(\mathbf{w}, b) = 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0$$

Ejemplo: Regularización LASSO

En regresión, LASSO minimiza el error cuadrático pero impone una restricción en la norma L_1 de los pesos para promover la rareza (*sparsity*).

Formulación:

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{y} - X\mathbf{w}\|_2^2$$

sujeto a la restricción de presupuesto:

$$g(\mathbf{w}) = \sum_{j=1}^n |w_j| - t \leq 0$$

Donde $t > 0$ es un hiperparámetro que controla qué tan estricta es la selección de variables.

Lagrangiano y la Función Característica χ

Para transformar un problema restringido en uno no restringido, podemos usar la función característica $\chi_{\mathcal{C}}(\boldsymbol{\theta})$ que penaliza infinitamente si salimos de la región factible $\mathcal{C}$:

$$\min_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) + \chi_{\mathcal{C}}(\boldsymbol{\theta}), \quad \text{donde } \chi_{\mathcal{C}}(\boldsymbol{\theta}) = \begin{cases} 0 & \text{si } \boldsymbol{\theta} \in \mathcal{C} \\ \infty & \text{si } \boldsymbol{\theta} \notin \mathcal{C} \end{cases}$$

Como $\chi_{\mathcal{C}}$ no es diferenciable, usamos los **Multiplicadores de Lagrange** para aproximar esta penalización mediante el Lagrangiano:

$$L(\boldsymbol{\theta}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = f(\boldsymbol{\theta}) + \sum_{i=1}^p \lambda_i h_i(\boldsymbol{\theta}) + \sum_{j=1}^m \mu_j g_j(\boldsymbol{\theta})$$

De la Penalización Exacta al Lagrangiano

El problema restringido $\min f(\boldsymbol{\theta}) + \chi_C(\boldsymbol{\theta})$ es matemáticamente exacto pero intratable, ya que χ_C es una barrera “infinitamente empinada”:

$$\chi_C(\boldsymbol{\theta}) = \sup_{\boldsymbol{\mu} \geq 0, \boldsymbol{\lambda}} \left(\sum_{i=1}^p \lambda_i h_i(\boldsymbol{\theta}) + \sum_{j=1}^m \mu_j g_j(\boldsymbol{\theta}) \right)$$

¿Qué sucede aquí?

Si $\boldsymbol{\theta}$ es factible: El valor máximo de la suma es 0 (cuando $h_i = 0$ y $g_j \leq 0$).

Si $\boldsymbol{\theta}$ es no factible: Si algún $g_j > 0$, podemos hacer $\mu_j \rightarrow \infty$ para que la suma sea ∞ .

La Relajación de Lagrange

El Lagrangiano $L(\boldsymbol{\theta}, \boldsymbol{\lambda}, \boldsymbol{\mu})$ reemplaza esta barrera de “todo o nada” por una **penalización lineal** controlada por los multiplicadores. Ya no forzamos el ∞ , sino que buscamos el equilibrio donde los multiplicadores reflejan el costo de violar la restricción.

Visión Dual: El Límite Inferior

Al intercambiar el orden del min y el sup, obtenemos la relación fundamental entre el Primal y el Dual:

$$\underbrace{\sup_{\lambda, \mu \geq 0} \inf_{\theta} L(\theta, \lambda, \mu)}_{\text{Problema Dual } (d^*)} \leq \underbrace{\inf_{\theta} \sup_{\lambda, \mu \geq 0} L(\theta, \lambda, \mu)}_{\text{Problema Primal } (p^*)}$$

El **Primal** es el problema original “difícil” con la barrera χ .

El **Dual** es maximizar la función dual $q(\lambda, \mu)$, que es siempre cóncava y más fácil de optimizar.

$\Rightarrow$ Esta estructura es la que permite que algoritmos como SVM resuelvan el problema en el espacio de los multiplicadores μ_i en lugar de los pesos w .

La Función Dual de Lagrange

Definimos la **Función Dual** $q(\lambda, \mu)$ como el valor mínimo del Lagrangiano respecto a la variable original θ :

$$q(\lambda, \mu) = \inf_{\theta \in \mathbb{R}^n} L(\theta, \lambda, \mu)$$

Propiedades Clave:

La función dual q es **siempre cóncava**, incluso si el problema original no es convexo.

Provee una cota inferior para el valor mínimo del problema original.

Solución Factible y el Lema Dual

Sea θ cualquier punto **factible** (que cumple $h_i = 0$ y $g_j \leq 0$) y sean $\mu \geq 0$ y λ multiplicadores arbitrarios.

Lema (Dualidad Débil): Para todo θ factible y $\mu \geq 0$:

$$q(\lambda, \mu) \leq f(\theta)$$

Prueba rápida:
$$L(\theta, \lambda, \mu) = f(\theta) + \underbrace{\sum \lambda_i h_i(\theta)}_0 + \underbrace{\sum \mu_j g_j(\theta)}_{\leq 0} \leq f(\theta).$$

Como q es el ínfimo de L sobre todo θ , entonces $q(\lambda, \mu) \leq L \leq f(\theta)$.

Problema Dual y Condiciones de Slater

El **Problema Dual** consiste en encontrar la mejor cota inferior posible:

$$\max_{\lambda, \mu \geq 0} q(\lambda, \mu)$$

Dualidad Fuerte y Condición de Slater: Decimos que hay Dualidad Fuerte si el máximo del dual es igual al mínimo del primal ($q^* = f^*$). Esto ocurre si el problema es convexo y se cumple la **Condición de Slater**:

Existe al menos un punto θ tal que todas las restricciones de desigualdad se cumplen estrictamente: $g_j(\theta) < 0$ para todo j .

Motivación: ¿Por qué KKT?

Si se cumple la condición de Slater (dualidad fuerte), sabemos que existe un punto donde el valor del primal coincide con el dual: $f(\theta^*) = q(\lambda^*, \mu^*)$.

Las **condiciones de Karush-Kuhn-Tucker (KKT)** son el conjunto de ecuaciones que **caracterizan** ese punto óptimo.

Son **necesarias** para cualquier problema de optimización diferenciable.

Son **suficientes** si el problema es además convexo.

⇒ Encontrar el óptimo restringido equivale a resolver el sistema KKT.

Las Condiciones KKT: Paso a Paso (I)

Sea θ^* un mínimo local y (λ^*, μ^*) los multiplicadores óptimos. Se deben cumplir cuatro bloques de condiciones:

1. Estacionariedad (Balance de Fuerzas): El gradiente del Lagrangiano respecto a θ debe anularse:

$$\nabla f(\theta^*) + \sum_{i=1}^p \lambda_i^* \nabla h_i(\theta^*) + \sum_{j=1}^m \mu_j^* \nabla g_j(\theta^*) = \mathbf{0}$$

Intuición: El gradiente de la función objetivo se cancela exactamente con la suma ponderada de los gradientes de las restricciones.

2. Factibilidad Primal (Respeto a las Restricciones): El punto debe estar en el conjunto factible:

$$h_i(\theta^*) = 0 \quad \text{y} \quad g_j(\theta^*) \leq 0$$

Las Condiciones KKT: Paso a Paso (II)

3. Factibilidad Dual (Signo de los Multiplicadores): Para las desigualdades, los multiplicadores deben ser no negativos:

$$\mu_j^* \geq 0, \quad j = 1, \dots, m$$

Intuición: Esto asegura que la restricción solo pueda “empujar” hacia el interior de la región factible.

4. Holgura Complementaria (La condición clave): El producto del multiplicador y su restricción debe ser cero:

$$\mu_j^* g_j(\theta^*) = 0, \quad \forall j$$

Implicación:

Si la restricción es **inactiva** ($g_j < 0$), entonces el multiplicador **debe ser** $\mu_j = 0$.

Si el multiplicador es **activo** ($\mu_j > 0$), entonces la restricción **debe ser** una

Resumen: Las 4 Condiciones KKT

Para un punto θ^* y multiplicadores (λ^*, μ^*) , las condiciones de Karush-Kuhn-Tucker son:

Condición	Expresión Matemática
1. Estacionariedad	$\nabla f(\theta^*) + \sum \lambda_i^* \nabla h_i(\theta^*) + \sum \mu_j^* \nabla g_j(\theta^*) = 0$
2. Factibilidad Primal	$h_i(\theta^*) = 0 \quad y \quad g_j(\theta^*) \leq 0$
3. Factibilidad Dual	$\mu_j^* \geq 0$
4. Holgura Compl.	$\mu_j^* g_j(\theta^*) = 0$

Estas condiciones vinculan el **Primal** con el **Dual**.

Son la base para identificar los vectores de soporte en SVM ($\mu_j > 0$).

Teorema de Karush-Kuhn-Tucker (KKT)

Sea f, g_j funciones convexas y h_i funciones afines, todas diferenciables.

Teorema KKT

Si se cumple la **Condición de Slater**, entonces θ^* es un **óptimo global** si y sólo si existen multiplicadores (λ^*, μ^*) que satisfacen las 4 condiciones KKT.

Puntos clave para recordar:

Necesidad: Para cualquier problema diferenciable, si θ^* es mínimo, entonces **deben** existir tales multiplicadores.

Suficiencia: Si el problema es **convexo**, cualquier punto que cumpla KKT es automáticamente el mínimo global.

Dualidad Fuerte: Bajo estas condiciones, el salto de dualidad es cero ($p^* = d^*$).

Teorema de Equivalencia KKT

En el contexto de modelos lineales (como SVM y LASSO), donde las restricciones suelen ser afines, disponemos de un resultado de equivalencia fundamental:

Teorema de Optimalidad

Si f es una función convexa y las funciones g_i son afines para $i \in \{1, \dots, m\}$, entonces los siguientes puntos son equivalentes:

- 1) $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$ satisfacen las condiciones KKT.
- 2) $\mathbf{x}^*$ es el óptimo del problema primal y $\boldsymbol{\lambda}^*$ es el óptimo del problema dual.

¿Por qué es tan importante para nosotros?

Garantía de Solución: Nos asegura que no hay un “salto de dualidad” (gap). Resolver el dual es, sin ambigüedades, resolver el problema original.

Implementación: En algoritmos como SVM, esto justifica por qué podemos

Minimización cuadrática con restricciones lineales

Minimización cuadrática con restricciones lineales

Problema ilustración. Considere el problema de minimizar la función cuadrática

$$F(\mathbf{x}) = x_1^2 + x_2^2, \quad \text{sobre la línea } K : a_1x_1 + a_2x_2 = b.$$

Multiplicadores de Lagrange:

Multiplicamos la restricción lineal $a_1x_1 + a_2x_2 = b$ por un multiplicador (desconocido) λ y lo agregamos a la función objetivo $F(x)$.

Definimos la nueva función **Lagrangeano**:

$$L(\mathbf{x}, \lambda) = F(\mathbf{x}) + \lambda(a_1x_1 + a_2x_2 - b)$$

Calculamos las derivadas de L e igualamos a cero.

Resolvemos las ecuaciones para x_1, x_2, λ .

Resolvemos

$$\frac{\partial L}{\partial x_1} = 2x_1 + \lambda a_1 = 0 \quad \longrightarrow \quad x_1 = -\lambda \frac{a_1}{2}$$

$$\frac{\partial L}{\partial x_2} = 2x_2 + \lambda a_2 = 0 \quad \longrightarrow \quad x_2 = -\lambda \frac{a_2}{2}$$

$$\frac{\partial L}{\partial \lambda} = a_1 x_1 + a_2 x_2 - b = 0 \quad \longrightarrow \quad \lambda = -\frac{2b}{a_1^2 + a_2^2}$$

Por lo tanto, el punto solución $\mathbf{x}^* = (x_1^*, x_2^*)$ y el costo mínimo $F(\mathbf{x}^*)$ son:

$$x_1^* = \frac{a_1 b}{a_1^2 + a_2^2}, \quad x_2^* = \frac{a_2 b}{a_1^2 + a_2^2}, \quad F(\mathbf{x}^*) = \frac{b^2}{a_1^2 + a_2^2}.$$

Además, la derivada del costo mínimo respecto al nivel de la restricción b es:

$$\frac{d}{db} F(\mathbf{x}^*) = \frac{d}{db} \left(\frac{b^2}{a_1^2 + a_2^2} \right) = \frac{2b}{a_1^2 + a_2^2} = -\lambda.$$

Minimización cuadrática con restricciones lineales

Sea $S \in \mathbb{R}^{n \times n}$ una matriz simétrica y definida positiva. Sean $A \in \mathbb{R}^{n \times m}$ y $b \in \mathbb{R}^m$. Consideramos el siguiente problema:

Minimización cuadrática

$$\text{minimizar } F(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top S \mathbf{x} \quad , \text{ sujeto a } A^\top \mathbf{x} = b$$

En este caso, tenemos m -multiplicadores $\lambda = [\lambda_1, \dots, \lambda_m]^\top$ y el Lagrangeano se define por:

$$L(\mathbf{x}, \lambda) = \frac{1}{2} \mathbf{x}^\top S \mathbf{x} + \lambda^\top (A^\top \mathbf{x} - b).$$

Calculamos las derivadas de L e igualamos a cero, obteniendo $n + m$ ecuaciones

$$\frac{\partial L}{\partial \mathbf{x}} = S \mathbf{x} + A \lambda = 0, \quad \frac{\partial L}{\partial \lambda} = A^\top \mathbf{x} - b = 0$$

Minimización cuadrática con restricciones lineales

La solución de estas ecuaciones $(\mathbf{x}^*, \lambda^*)$ satisface las siguiente ecuaciones:

$$\begin{bmatrix} S & A \\ A^\top & 0 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \lambda \end{bmatrix} = \begin{bmatrix} 0 \\ b \end{bmatrix}$$

Esta matriz se conoce como la matriz de Karusch-Kuhn-Tucker. El problema matricial se expresa en términos de su complemento de Schur como

$$\begin{bmatrix} S & A \\ 0 & -A^\top S^{-1} A \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \lambda \end{bmatrix} = \begin{bmatrix} 0 \\ b \end{bmatrix}$$

De donde la solución se escribe:

$$\lambda^* = -(A^\top S^{-1} A)^{-1} b, \quad \mathbf{x}^* = S^{-1} A (A^\top S^{-1} A)^{-1} b$$

Solución

Así el costo mínimo en el punto solución queda:

$$F(\mathbf{x}^*, \lambda^*) = \frac{1}{2} b^\top (A^\top S^{-1} A)^{-1} b$$

y el gradiente del costo mínimo respecto al vector b es

$$\frac{\partial F}{\partial b}(\mathbf{x}^*, \lambda^*) = (A^\top S^{-1} A)^{-1} b = -\lambda^*.$$

Observación. La función $L = \frac{1}{2} \mathbf{x}^\top S \mathbf{x} + \lambda^\top (A^\top \mathbf{x} - b)$ es convexa en $\mathbf{x}$ y cóncava en λ .

Propiedad minmax/maxmin

Suponga que separamos dos problemas:

- 1 Minimizar $L(\mathbf{x}, \lambda)$ para λ fijo, y luego maximizar sobre λ

$$\max_{\lambda} \min_{\mathbf{x}} L(\mathbf{x}, \lambda) = \frac{1}{2} b^{\top} (A^{\top} S^{-1} A)^{-1} b$$

- 2 Maximizar $L(\mathbf{x}, \lambda)$ para $\mathbf{x}$ fijo y luego minimizar sobre $\mathbf{x}$

$$\min_{\mathbf{x}} \max_{\lambda} L(\mathbf{x}, \lambda) = \frac{1}{2} b^{\top} (A^{\top} S^{-1} A)^{-1} b$$

En el punto silla $(\mathbf{x}^*, \lambda^*)$ tenemos que :

$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \lambda} = 0 \quad \text{y} \quad \max_{\lambda} \min_{\mathbf{x}} L(\mathbf{x}, \lambda) = \min_{\mathbf{x}} \max_{\lambda} L(\mathbf{x}, \lambda)$$



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE